

TECHNOLOGY STACK

1. Technology Overview

The application uses a JavaScript-based full-stack web architecture. The backend is implemented using Node.js and Express, data is managed using MongoDB through Mongoose, and the frontend is delivered through HTML, CSS and JavaScript pages.

2. Technology Stack

Runtime	Node.js	Execute the backend JavaScript application.
Backend Framework	Express.js	Build REST API routes and middleware.
Database	MongoDB	Store users, categories and FAQs.
ODM	Mongoose	Define schemas and interact with MongoDB.
Authentication	JSON Web Token (JWT)	Authenticate users and protect APIs.
Password Security	bcryptjs	Hash user passwords.
Configuration	dotenv	Load environment configuration.
Cross-Origin	cors	Support controlled cross-origin API requests.
Frontend	HTML5 / CSS / JavaScript	Provide web pages and user interactions.
AI Logic	Custom JavaScript similarity engine	Match user questions with FAQ content.
Development	Node --watch	Support development-time server restart.
Testing Support	mongodb-memory-server dependency	Provide an in-memory MongoDB option for development/testing scenarios.

3. Application Components

Server	src/server.js	Starts the application server.
Application	src/app.js	Configures Express application and middleware/routes.
Database	src/config/db.js	Database connection configuration.
Controllers	src/controllers/	Business logic for authentication, FAQs, categories, users and AI.
Routes	src/routes/	REST endpoint definitions.
Middleware	src/middleware/	Authentication and centralized error handling.
Models	src/models/	User, Category and FAQ schemas.
AI Utility	src/utls/aiEngine.js	Text processing, scoring and FAQ matching.
Frontend Pages	public/*.html	Login, registration, dashboard, admin and AI assistant interfaces.

Frontend Scripts	public/js/*.js	Client-side authentication, dashboard, admin, main and AI interactions.

4. Data Entities

User	name, email, password, role, createdAt
Category	name, description, createdAt
FAQ	question, answer, category, keywords, views, createdAt, updatedAt

5. AI Matching Technology

The project uses a **custom JavaScript similarity engine** rather than an external generative AI API. It tokenizes the question, removes stop words, checks exact and near-exact matches, compares keywords and question/answer tokens, applies a coverage bonus, sorts candidate FAQs by score, and uses a configured threshold to decide whether a relevant result is available.

6. Technology Selection Rationale

REST API	Express.js	Lightweight and modular backend API development.
Document Database	MongoDB	Suitable for flexible FAQ and category data.
Data Modeling	Mongoose	Provides schemas, validation structure and MongoDB integration.
Authentication	JWT + bcryptjs	Supports token-based authentication and password hashing.
AI FAQ Matching	JavaScript utility	Keeps the matching mechanism integrated with the Node.js backend.
Web UI	HTML/CSS/JavaScript	Simple browser-based interface for students and administrators.

7. Development Dependency Summary

Production dependencies: bcryptjs, cors, dotenv, express, jsonwebtoken, mongoose.
Development dependency: mongodb-memory-server.